



Lab Title: Hands-on Secure Coding & Code Review

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS106 – Secure Coding Practices and Input Validation

Instructor Name: AMMAR AMINU

Date of Submission: 29/04/2026

Executive Summary

This report presents the findings and practical analysis from a secure coding lab focused on identifying and mitigating common web application vulnerabilities. The lab covered core security principles, vulnerability identification using CWE standards, and secure implementation practices in PHP and Python. Through code reviews and simulated exploitation scenarios, multiple weaknesses such as SQL Injection, Cross-Site Scripting (XSS), and improper session management were identified and remediated. The objective was to strengthen secure coding skills and improve the ability to design resilient applications.

Lab Objective

The objective of this lab is to develop practical skills in identifying, analyzing, and mitigating security vulnerabilities in web applications by applying secure coding principles and performing code reviews.

Part 1: Core Principles and Foundational Concepts

1.1 The CIA Triad

Defining the three pillars of the CIA Triad (Confidentiality, Integrity, and Availability).

- **Confidentiality**

Ensures data secrecy from unauthorized people

- **Integrity**

Ensures safeguard accuracy and completeness of data

- **Availability**

Ensures data accessibility from unauthorized user when need.

Linking each attack to the CIA Triad it affect

1 SQL Injection

Affected Pillars: Confidentiality and Integrity

Explanation: SQL injection allows attacker to access sensitive information by injecting a malicious payload and also modifying and deleting information from the database.

2 Denial of Service (DoS)

Affected Pillars: Availability

Explanation: DoS attack overwhelm the system, making it unavailable to the user.

3 Unauthorized Data Access

Affected Pillars: Confidentiality

Explanation: Unauthorized user gain access into sensitive information, confidentiality is violated.

1.2 Security Principles in Practice

Explain the following five secure coding principles, providing a distinct real-world software example for each to illustrate its application:

4 Minimize Attack Surface

Explanation: Reduce the number of entry points an attacker can exploit by disabling unnecessary features, services, or APIs.

Real-World Application: A web application disables unused API endpoints and removes legacy admin panels that are no longer needed, ensuring attackers have fewer ways to probe the system.

5 Establish Secure Defaults

Explanation: Applications should ship with the most secure settings enabled by default, requiring users to explicitly opt-in to weaker configurations if absolutely necessary.

Real-World Application: A cloud storage service (like AWS S3) defaults new buckets to private access, requiring explicit configuration to make them public. This prevents accidental data leaks.

6 Least Privilege

Explanation: Access should be limited to only what is necessary—no one, not even trusted users or accounts, should automatically have unrestricted access to everything.

Real-World Application: A database account used by a reporting tool is granted **read-only access** to specific tables, preventing it from modifying or deleting data.

7 Defense in Depth

Explanation: Single Security method should not be rely to protect everything, rather layer basic security practices to ensure the overall safety of an application.

Real-World Application: An online banking system uses multi-factor authentication (MFA), TLS encryption, firewalls, and intrusion detection systems together to protect accounts.

8 Fail Securely

Explanation: When security controls fail, they should default to a secure state — denying access until validation succeeds.

Real-World Application: A login system that encounters an error during authentication denies access by default rather than granting it, preventing accidental exposure of accounts.

1.3 Common Threats

Briefly describe the nature of the following common application security threats:

9 Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a vulnerability that allows an attacker to inject malicious HTML or JavaScript code (known as a payload) into a web application. When other users visit the affected site, the injected code executes in their browsers.

This can enable attackers to:

- Steal session cookies and impersonate users
- Spread malware through the site

- Deface web pages
- Replace legitimate pages with fake login forms to phish credentials

XSS ranks **third in the OWASP Top 10** under the Injection category.

A common scenario is when a web application accepts user input (like a comment form) and displays it without proper sanitization. For example, if an attacker submits:

```
<script>alert('Hacked!');</script>
```

and the site displays it directly, every visitor viewing that comment will see the popup. This demonstrates how unvalidated input can lead to arbitrary script execution in users' browsers.

10 Cross-Site Request Forgery (CSRF)

This vulnerability tricks a logged-in user's browser into sending unwanted requests to a web application without their knowledge. Because the browser automatically includes cookies or session tokens, the application thinks the request is legitimate.

- **Example:** If you're logged into your bank and visit a malicious site, it could secretly send a request to transfer money using your active session.
- **Impact:** Attackers can perform actions as you - like changing your password, making purchases, or transferring funds - all without your consent.

11 Broken Authentication and Session Management

This occurs when login systems or session handling are poorly implemented, making it easy for attackers to hijack accounts.

- **Example:** If passwords are weak, session IDs never expire, or tokens aren't properly invalidated, attackers can guess or steal credentials and stay logged in indefinitely.
- **Impact:** Attackers can impersonate users, escalate privileges, or maintain unauthorized access to sensitive systems.

12 Insecure Deserialization

This vulnerability happens when an application accepts serialized objects (data structures turned into strings) from untrusted sources and deserializes them without validation. Attackers can craft malicious objects that execute arbitrary code when loaded.

- **Example:** If a web app loads user-supplied serialized data directly, an attacker could inject code that runs on the server.
- **Impact:** Remote code execution, privilege escalation, or complete system compromise.

Part 2: CWE Identification and Mitigation Strategies

Review the descriptions of the vulnerabilities below and fill in the corresponding CWE ID and a brief, general mitigation strategy based on the course slides and common secure coding practices.

Vulnerability Description	CWE ID (e.g., CWE-89)	Mitigation Strategy
The software constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralize special elements that could modify the intended SQL command.	CWE-89 (SQL Injection)	Use parameterized queries or prepared statements; validate and sanitize all user input; apply least-privilege access to the database.
The product does not require users to have strong passwords, which makes it easier for attackers to compromise user accounts.	CWE-521 (Weak Password Requirements)	Enforce strong password policies-minimum length, complexity, and periodic rotation; encourage multi-factor authentication.
The software does not implement sufficient measures to prevent multiple failed authentication attempts within in a short time frame, making it more susceptible to brute force attacks.	CWE-307 (Improper Restriction of Excessive Authentication Attempts)	Implement account lockout or rate-limiting after repeated failures; use CAPTCHA or incremental delays.
The web application uses the GET method to process requests that contain sensitive information, which can expose that information through the browser's history, referers, web logs, and other sources.	CWE-598 (Use of GET Request for Sensitive Data)	Use POST for sensitive data; avoid exposing credentials or tokens in URLs; apply HTTPS to encrypt transmissions.
The lack of proper session expiration may improve the likely success of certain attacks.	CWE-613 (Insufficient Session Expiration)	Set appropriate session timeouts; invalidate sessions after logout or inactivity; regenerate session IDs periodically.
Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier allows an attacker to steal authenticated sessions.	CWE-384 (Session Fixation)	Regenerate session IDs after authentication; invalidate old sessions; use secure, random session tokens.
The software does not neutralize or incorrectly neutralizes user-controllable input before it is placed in output that is used as web page content, leading to the execution of malicious client-side scripts.	CWE-79 (Cross-Site Scripting [XSS])	Encode output, validate input, and use frameworks with built-in XSS protection; apply Content Security Policy (CSP)
The software does not sufficiently verify whether a request was	CWE-352	Use anti-CSRF tokens; validate origin and referer

intentionally created by the user, allowing an attacker to trick the user into performing actions they did not intend.	(Cross-Site Request Forgery [CSRF])	headers; apply same-site cookies.
The software does not properly restrict access to resources based on the user's privileges, allowing unauthorized users to perform actions or access data they should not.	CWE-284 (Improper Access Control)	Implement role-based access control; validate authorization on every request; follow least-privilege principle.
The software does not properly handle errors, leading to the disclosure of sensitive information or a denial of service.	CWE-209 (Information Exposure Through Error Message)	Use generic error messages; log detailed errors securely; avoid exposing stack traces or system details to users.

Part 3: Authentication and Password Management

3.1 Theory Questions

13 Difference Between Authentication and Authorization

- **Authentication** verifies who a user is - confirming identity through credentials like username and password.
- **Authorization** determines what that authenticated user is allowed to do - defining access rights and permissions.
- **Why separation matters:** Keeping these concerns distinct prevents privilege escalation and enforces the least privilege principle. It ensures that even if authentication succeeds, users can only access resources they're authorized for, reducing the attack surface.

14 Three Primary Factors of Authentication

Factor	Description	Example
Something you know	Information only the user should know.	Password, PIN, or security question.
Something you have	A physical or digital item used for verification.	Smart card, mobile token, or one-time code sent to a phone.
Something you are	Biometric characteristics unique to the user.	Fingerprint, facial recognition, or iris scan.

15 Security Implications of a Weak Password Recovery Mechanism

A weak recovery process (e.g., predictable questions or insecure reset links) can let attackers bypass authentication entirely. **Risks:** Account takeover, exposure of sensitive data, and compromised system integrity. Secure Password Reset Flow:

- Require identity verification through secure channels (e.g., email or SMS with time-limited tokens).
- Use HTTPS for all reset communications.
- Invalidate old tokens immediately after use.
- Enforce strong new-password policies and log reset attempts for auditing.

These practices align with the Fail Securely and Establish Secure Defaults principles emphasized in your slides.

3.2 PHP Code Review: Vulnerable Authentication

Review the following PHP code snippet for a user login function. Identify all security vulnerabilities and propose specific remediations.

```
<?php
```

```
// login.php
include("db_config.php");

$username = $_REQUEST["username"]; // Using $_REQUEST for flexibility
$password = $_REQUEST["password"];

// Query to check username and password
$sql = "SELECT * FROM users WHERE username = ".$username." AND password = ".md5($password);
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    $row = $result->fetch_assoc();
    session_start();
    $_SESSION["loggedin"] = true;
    $_SESSION["id"] = $row["id"];
    $_SESSION["username"] = $row["username"];
    $_SESSION["role"] = $row["role"];
    header("Location: welcome.php");
} else {
    echo "<p>Invalid username or password.</p>";
}

$conn->close();
```

```
?>
```

Practical Tasks:

16 Identify Vulnerabilities

Vulnerability	CWE ID	Description
SQL Injection	CWE-89	The code directly concatenates user input into SQL queries without sanitization or prepared statements, allowing attackers to manipulate queries.
Plaintext Password Storage	CWE-256	Passwords are stored or compared in plaintext rather than using secure hashing, exposing credentials if the database is compromised.
Improper Error Handling	CWE-209	The script displays raw error messages (e.g., "Invalid username or password") that can reveal system behavior or valid usernames.
Session Fixation / Improper Session Management	CWE-384	The code fails to regenerate session IDs after login, allowing attackers to hijack sessions.

17 Exploitation Scenario:

SQL Injection Attack An attacker could exploit the SQL Injection vulnerability by entering malicious input into the login form.

Payload Example:

sql

' OR '1'='1

How it works: When the code executes the query without sanitization, the injected condition ('1'='1') always evaluates to true, bypassing authentication and granting unauthorized access to any account.

18 Remediation Plan:

A secure step-by-step plan to fix all vulnerabilities:

1. **Use Prepared Statements (PDO or MySQLi):** Replace dynamic SQL concatenation with parameterized queries to prevent SQL Injection.
2. **Implement Password Hashing:** Use `password_hash()` and `password_verify()` for secure password storage and comparison.
3. **Validate and Sanitize Input:** Apply server-side validation and escape special characters before processing user data.
4. **Secure Session Management:** Regenerate session IDs after login, set secure cookie flags (HttpOnly, Secure), and implement proper session expiration.
5. **Generic Error Messages:** Display non-specific messages to users while logging detailed errors securely on the server.
6. **Enforce Strong Password Policy:** Require complex passwords and limit failed login attempts to mitigate brute-force attacks.
7. **Use HTTPS:** Encrypt all communication between client and server to protect credentials in transit.

3.3 Python Code Review: Password Reset Flaw

Consider the following Python Flask snippet for a password reset functionality. Identify the security flaw and suggest a secure remediation.

```
from flask import Flask, request, redirect, url_for, render_template_string
```

```
import sqlite3
```

```
app = Flask(__name__)
```

```
@app.route("/reset_password", methods=["GET", "POST"])
```

```
def reset_password():
```

```
    if request.method == "POST":
```

```
        email = request.form.get("email")
```

```
        new_password = request.form.get("new_password")
```

```
        confirm_password = request.form.get("confirm_password")
```

```
        if new_password != confirm_password:
```

```

    return "Passwords do not match!"

conn = sqlite3.connect("users.db")
cursor = conn.cursor()
# Vulnerable: No token validation, relies solely on email
cursor.execute(f"UPDATE users SET password = '{new_password}' WHERE email = '{email}'")
conn.commit()
conn.close()
return "Password reset successfully!"
return render_template_string("""
<form method="post">
    Email: <input type="text" name="email"><br>
    New Password: <input type="password" name="new_password"><br>
    Confirm Password: <input type="password" name="confirm_password"><br>
    <input type="submit" value="Reset Password">
</form>
""")

if __name__ == "__main__":

app.run(debug=True)

```

Practical Tasks:

19 Identify Vulnerability:

The primary flaw in the Flask password-reset snippet is that it does not use secure, unique, time-limited tokens to verify the identity of the user requesting a password reset. Instead, it likely resets passwords directly based on a username or email parameter without proper validation.

Relevant CWE: CWE-640 (Weak Password Recovery Mechanism for Forgotten Password) - the application allows password resets without strong authentication or token validation.

20 Exploitation Scenario:

An attacker could exploit this flaw by submitting a crafted request that includes another user's email or username.

Example:

http

POST /reset_password

Content-Type: application/x-www-form-urlencoded

email=victim@example.com&new_password=Attacker123

Because the code doesn't verify ownership of the email or require a secure token, the attacker can change the victim's password and gain full access to their account.

21 Remediation:

To implement a secure password-reset mechanism:

1. **Generate Unique, Time-Limited Tokens:**
 - When a user requests a reset, create a cryptographically secure random token (e.g., using `secrets.token_urlsafe()` in Python).
 - Store the token with an expiration timestamp in the database.
2. **Send Token via Registered Email:**
 - Email the token link only to the user's registered address (e.g., `https://example.com/reset?token=<token>`).
 - Never expose tokens in logs or URLs accessible to others.
3. **Validate Token on Reset Page:**
 - When the user clicks the link, verify the token's validity and expiration before allowing password change.
4. **Use Secure Password Handling:**
 - Hash new passwords using `bcrypt` or `werkzeug.security.generate_password_hash()`.
 - Invalidate the token immediately after successful reset.
5. **Apply HTTPS and Generic Responses:**
 - Use HTTPS for all reset communications.
 - Provide generic messages like "If your email exists, a reset link has been sent" to avoid user enumeration.

Part 4: Authorization and Access Control

4.1 Theory Questions

22 Least Privilege

Concept: Least Privilege means every user, process, or system component should have only the minimum permissions necessary to perform its tasks - nothing more.

Why it's fundamental:

- Reduces the impact of compromised accounts or applications.
- Limits unauthorized access to sensitive data.
- Supports defense-in-depth by minimizing the attack surface.

In application design, separating privileges ensures that even if one layer is breached, attackers cannot escalate their access beyond what's strictly required.

23 Difference Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC).

Model	Description	Example Scenario
Role-Based Access Control (RBAC)	Access is granted based on predefined roles (e.g., Admin, Editor, and Viewer). Each role has fixed permissions.	A content-management system where editors can modify posts and viewers can only read them.

Attribute-Based Access Control (ABAC)	Access decisions are based on attributes of the user, resource, and environment (e.g., department, clearance level, time, location).	A healthcare system that allows doctors to view patient records only during working hours and only for patients in their department.
---------------------------------------	--	--

When ABAC is more suitable:

ABAC is ideal for dynamic environments requiring fine-grained control for example, cloud applications where access depends on multiple contextual factors rather than static roles.

24 What is Horizontal Privilege Escalation and Vertical Privilege Escalation? Provide an example for each.

Type	Description	Example
Horizontal Privilege Escalation	A user gains access to another user's data or functions at the same privilege level.	A regular user modifies a URL parameter (/profile?id=2) to view another user's profile.
Vertical Privilege Escalation	A user gains higher-level privileges than intended.	A normal user exploits a flaw to access admin-only pages or perform administrative actions.

Both types violate the Least Privilege principle and highlight the need for strict authorization checks on every request.

4.2 PHP Code Review: Insecure Direct Object Reference (IDOR)

Consider a PHP application that allows users to view their own profile details. The following code snippet is used to retrieve a user's profile.

```
<?php
```

```
// view_profile.php
include("db_config.php");
session_start();

$user_id = $_GET["id"]; // User ID from URL parameter

// Assume $_SESSION["user_id"] holds the currently logged-in user's ID
```

```
// if (!isset($_SESSION["user_id"])) {
//     header("Location: login.php");
//     exit();
// }

$sql = "SELECT * FROM profiles WHERE user_id = ".$_user_id;
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    $profile = $result->fetch_assoc();
    echo "<h1>Profile for " . $profile["name"] . "</h1>";
    echo "<p>Email: " . $profile["email"] . "</p>";
    echo "<p>Bio: " . $profile["bio"] . "</p>";
} else {
    echo "Profile not found.";
}

$conn->close();
```

```
?>
```

Practical Tasks:

25 Identify Vulnerability:

The primary vulnerability is Insecure Direct Object Reference (IDOR) - the application directly uses user-supplied identifiers (like `user_id` in a query string) to access resources without verifying authorization.

Relevant CWE: CWE-639 (Authorization Bypass Through User-Controlled Key) or CWE-285 (Improper Authorization).

Nature: The code trusts user input to determine which profile to display, allowing attackers to manipulate identifiers and access data they shouldn't.

26 Exploitation Scenario: Describe how an attacker could view another user's profile without proper authorization.

An attacker could exploit this by changing the `user_id` parameter in the URL or form data.

Example:

http

GET /profile.php?user_id=102

If the attacker's own ID is 101, but the application doesn't check authorization, they can view another user's profile (102) simply by altering the parameter. This is a horizontal privilege escalation - accessing another user's data at the same privilege level.

27 Remediation:

To secure the PHP code and enforce proper authorization:

1. **Use Session-Based Identification:** Retrieve the logged-in user's ID from the session rather than trusting external input.

php

```
$user_id = $_SESSION['user_id'];
```

2. **Implement Authorization Checks:** Verify that the requested resource belongs to the authenticated user or that the user has the necessary role (e.g., admin).

php

```
if ($requested_id != $_SESSION['user_id'] && $_SESSION['role'] != 'admin') {  
    die('Unauthorized access');  
}
```

3. **Validate Input:** Sanitize and validate all parameters before using them in queries.
4. **Use Least Privilege Principle:** Ensure users can only access their own data unless explicitly authorized.
5. **Apply Secure Error Handling:** Display generic error messages and log unauthorized attempts for auditing.

4.3 Python Code Review: Insecure Authorization Logic

Consider a Python Flask application with an admin panel. The following code snippet attempts to restrict access to administrators.

```
from flask import Flask, request, redirect, url_for, session, render_template_string
```

```
app = Flask(__name__)  
app.secret_key = "super_secret_key"  
  
@app.route("/admin_panel")  
def admin_panel():  
    # Assume session["user_role"] is set after login  
    if session.get("user_role") == "admin":  
        return "<h1>Welcome to the Admin Panel!</h1><p>Sensitive administrative functions here.</p>"  
    else:  
        return redirect(url_for("login")) # Redirect to login if not admin  
  
@app.route("/login", methods=["GET", "POST"])  
def login():  
    if request.method == "POST":  
        username = request.form.get("username")  
        password = request.form.get("password")  
        # Simplified login for demonstration  
        if username == "admin" and password == "password123":  
            session["user_role"] = "admin"  
            return redirect(url_for("admin_panel"))
```

```

elif username == "user" and password == "password":
    session["user_role"] = "user"
    return redirect(url_for("user_dashboard"))
else:
    return "Invalid credentials"
return render_template_string("""
<form method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>
""")

@app.route("/user_dashboard")
def user_dashboard():
    if session.get("loggedin"): # Simplified check
        return "<h1>Welcome to your Dashboard!</h1>"
    return redirect(url_for("login"))

if __name__ == "__main__":

    app.run(debug=True)

```

Practical Tasks:

28 Identify Vulnerability:

The potential weakness lies in how the `admin_panel` function checks for administrative privileges. If the code relies solely on a session variable (e.g., if `session['role'] == 'admin':`) without re-validating the user's role from a trusted source like the database, it violates the "Don't Trust Services" principle. Relevant CWE: **CWE-285 (Improper Authorization)** - the application trusts client-side or session data for access control instead of verifying it securely on the server.

Nature: Attackers could manipulate session data or cookies to impersonate an admin.

29 Exploitation Scenario:

A non-admin user could exploit this by tampering with their session data.

Example: If the session is stored client-side or poorly protected, an attacker could modify it using browser tools or intercept traffic to change:

Python

```
Session['role'] = 'admin'
```

After altering the role, the attacker gains unauthorized access to the admin panel without legitimate admin credentials - a vertical privilege escalation.

30 Remediation:

To make the authorization check more robust:

1. **Re-validate Roles from a Trusted Source:** Query the database or a secure identity provider each time an admin-level action is requested.

Python

```
user = db.get_user(session['user_id'])
if user.role != 'admin':
    abort(403)
```

2. **Use Server-Side Session Management:** Store session data securely on the server; never rely on client-side modifications.
3. **Implement Role-Based Access Control (RBAC):** Define roles and permissions centrally, and enforce them consistently across all routes.
4. **Apply Least Privilege Principle:** Restrict admin access only to verified users and log all administrative actions.
5. **Use Secure Cookies:** Set HttpOnly, Secure, and SameSite flags to prevent tampering or theft.

Part 5: Session Management

5.1 Theory Questions

- 31** What is **Session Fixation**? Describe the attack flow and how it can be mitigated.

Definition: Session Fixation occurs when an attacker sets or predicts a user's session ID before the user logs in. Once the victim authenticates, the attacker can reuse that same session ID to hijack the session.

Attack Flow:

1. Attacker obtains or sets a known session ID.
2. Victim logs in using that session ID.
3. Attacker reuses the ID to access the victim's authenticated session.

Mitigation:

- Regenerate session IDs after successful login.
- Use secure, random, unpredictable session IDs.
- Set cookies with HttpOnly and Secure flags.

- 32** Explain the importance of session expiration. What are the risks of having sessions that never expire or expire too slowly?

Concept: Session expiration ensures that sessions automatically end after a period of inactivity or a fixed lifetime.

Risks of Never-Expiring Sessions:

- Increased exposure to hijacking if a user forgets to log out.
- Attackers can reuse old session IDs.
- Sensitive data remains accessible indefinitely.

Mitigation:

- Implement idle timeouts and absolute expiration limits.
- Require re-authentication for critical actions.
- Invalidate sessions on logout or after password changes.

33 What is a **Session ID**? What characteristics should a secure Session ID possess?

Definition: A Session ID is a unique identifier assigned by the server to track a user's session state.

Secure Characteristics:

- **Uniqueness:** Each session ID must be distinct.
- **Unpredictability:** Generated using cryptographically secure random functions.
- **Confidentiality:** Transmitted only over HTTPS; never exposed in URLs or logs.
- **Integrity:** Protected with HttpOnly and Secure cookie attributes to prevent theft or tampering.
- **Limited Lifetime:** Expire after logout or inactivity.

5.2 PHP Code Review: Session Fixation Vulnerability

Examine the following PHP login process. Identify the session management vulnerability.

```
<?php
```

```
// login_process.php
include("db_config.php");
session_start(); // Session started before authentication

$username = $_POST["username"];
$password = $_POST["password"];

// Assume successful authentication for demonstration
if ($username == "admin" && $password == "password") {
    $_SESSION["loggedin"] = true;
    $_SESSION["username"] = $username;
    header("Location: dashboard.php");
    exit();
} else {
    echo "Invalid credentials.";
}
```

```
?>
```

Practical Tasks:

34 Identify Vulnerability: What session management vulnerability is present in this code? Reference the relevant CWE.

The PHP login process starts a session before authentication and continues using the same session ID after login without regenerating it. This exposes the application to a Session Fixation vulnerability.

Relevant CWE: CWE-384 (Session Fixation) - the software does not invalidate or regenerate session IDs upon authentication, allowing attackers to hijack sessions.

35 Exploitation Scenario: Describe how an attacker could exploit this to hijack a legitimate user's session.

An attacker could exploit this by forcing a victim to use a known session ID.

Example Attack Flow:

1. Attacker starts a session and obtains a valid session ID.
2. Attacker sends a crafted link to the victim containing that session ID (e.g., <https://example.com/login.php?PHPSESSID=ABC123>).
3. Victim logs in using that session.
4. Because the application doesn't regenerate the session ID after login, the attacker can reuse [ABC123](#) to access the victim's authenticated session.

This allows full account takeover without needing the victim's credentials.

36 Remediation: Modify the PHP code to prevent this vulnerability.

To prevent Session Fixation, modify the PHP code to regenerate the session ID after successful authentication and apply secure session practices:

php

```
<?php
```

```
include("db_config.php");
```

```
session_start();
```

```
$username = $_POST['username'];
```

```
$password = $_POST['password'];
```

```
// Example authentication logic
```

```
if ($username === "admin" && $password === "password") {
```

```
    // Regenerate session ID after successful login
```

```
    session_regenerate_id(true);
```

```
    $_SESSION['loggedin'] = true;
```

```
    $_SESSION['username'] = $username;
```

```
    echo "Login successful!";
```

```
} else {
```

```
    echo "Invalid credentials.";
```

```
}  
?>
```

Additional Mitigations:

- Use secure, random session IDs.
- Set cookies with HttpOnly, Secure, and SameSite attributes.
- Implement proper session expiration and logout handling.
- Use HTTPS to protect session data in transit.

5.3 Python Code Review: Insecure Session Handling

Consider a Python Flask application where a user's session is managed as follows:

```
from flask import Flask, session, redirect, url_for, escape, request, render_template_string
```

```
app = Flask(__name__)  
app.secret_key = "insecure_secret_key_for_demo"  
  
@app.route("/set_session")  
def set_session():  
    if "username" in request.args:  
        session["username"] = request.args["username"]  
        return f"Session username set to {escape(session["username"])}"  
    return "No username provided"  
  
@app.route("/get_session")  
def get_session():  
    if "username" in session:  
        return f"Current session username is {escape(session["username"])}"  
    return "No username in session"  
  
if __name__ == "__main__":  
  
    app.run(debug=True)
```

Practical Tasks:

37 Identify Vulnerability: What is the critical security vulnerability related to app.secret_key in this Flask application? What are the implications of this vulnerability?

The critical vulnerability lies in how the Flask application handles `app.secret_key`. If the `secret_key` is hard-coded, weak, or publicly exposed, attackers can predict or forge session cookies. Relevant CWE: CWE-321 (Use of Hard-Coded Cryptographic Key).

Implications:

- Attackers can decrypt or tamper with session data.
- They can impersonate users or escalate privileges.

- Entire session integrity and confidentiality are compromised.

38 Remediation: How should app.secret_key be generated and managed in a production environment?

In a production environment, `app.secret_key` must be securely generated and managed:

- **Generate Securely:** Use a cryptographically strong random generator, e.g.:
 - `python`
 - `import secrets`
 - `app.secret_key = secrets.token_hex(32)`
- **Manage Securely:**
 - Store the key in environment variables or a secure configuration file, not in source code.
 - Rotate keys periodically.
 - Protect configuration files with proper access controls.
 - Never commit keys to version control systems (Git, etc.).

39 Additional Vulnerability: Identify another potential session-related vulnerability in the set_session route (Hint: how is the username being set?).

In the `set_session` route, if the username is set directly from user input (e.g., `session['username'] = request.args['username']` or `request.form['username']`), it introduces Insecure Direct Session Manipulation. Relevant CWE: CWE-602 (Client-Side Enforcement of Server-Side Security) **or** CWE-285 (Improper Authorization).

Risk: Attackers can craft requests to set arbitrary usernames in their session, impersonating other users without authentication.

Remediation:

- Only set session data after verifying credentials against a trusted source (e.g., database).
- Never trust client-supplied data for authentication or identity.
- Validate and sanitize all input before storing it in the session.

Part 6: Data Validation and Input Sanitization

6.1 Theory Questions

40 Explain the principle of "Never Trust User Input." Why is this fundamental to secure coding?

Explanation: All data coming from users, browsers, APIs, or external systems should be treated as untrusted until validated. Attackers can manipulate input to inject malicious code, corrupt data, or bypass logic.

Why it's fundamental:

- Prevents injection attacks (SQL Injection, XSS, and Command Injection).
- Ensures application integrity and reliability.

- Enforces the **Validate Input** and **Fail Securely** principles from secure-coding practices.

Key Idea: Always validate, sanitize, and encode user input before processing or storing it.

41 Differentiate between **Whitelisting** and **Blacklisting** for input validation. Which method is generally preferred and why?

Method	Description	Preferred Approach
Whitelisting	Defines a list of allowed input values or patterns (e.g., only letters, numbers, or specific formats).	Preferred - proactive and precise; allows only known-safe data.
Blacklisting	Blocks known bad input patterns (e.g., <code><script></script></code> or SQL keywords).	Less reliable - attackers can bypass filters with new or obfuscated payloads.

Why Whitelisting is Preferred: It enforces strict validation rules, reducing the risk of unexpected or malicious input slipping through.

42 What is the purpose of **Output Encoding**? Provide an example of where it should be used.

Definition: Output Encoding transforms data before displaying it so that special characters are treated as text, not executable code.

Purpose: Prevents **Cross-Site Scripting (XSS)** and other injection attacks by ensuring user-supplied data cannot alter HTML, JavaScript, or SQL contexts.

Example: If a user enters `<script>alert('XSS')</script>`, output encoding converts it to:

```
html
&lt;script&gt;alert('XSS')&lt;/script&gt;
```

The browser displays it as harmless text instead of executing it.

6.2 PHP Code Review: SQL Injection and XSS

Review the following PHP script that takes user input and displays it.

```
<?php
```

```
// guestbook.php
```

```

include("db_config.php");

if (isset($_POST["message"])) {
    $message = $_POST["message"];
    $username = $_POST["username"];

    // Vulnerable: No input validation or sanitization
    $sql = "INSERT INTO guestbook (username, message) VALUES ('.$username.', ".$message.")";
    $conn->query($sql);
}

// Display messages
$sql_select = "SELECT username, message FROM guestbook ORDER BY id DESC";
$result_select = $conn->query($sql_select);

while ($row = $result_select->fetch_assoc()) {
    echo "<p><b>". $row["username"] . " :</b> " . $row["message"] . "</p>";
}

$conn->close();

?>

```

Practical Tasks:

43 Identify Vulnerabilities: Identify at least **two** major vulnerabilities in this guestbook.php script. Reference the relevant CWEs.

The guestbook.php script likely takes user input (e.g., name and message) and inserts it directly into a database and then displays it back on the page. Two major vulnerabilities are present:

Vulnerability	CWE ID	Description
SQL Injection	CWE-89	User input is concatenated directly into SQL queries without sanitization or prepared statements, allowing attackers to manipulate queries.
Cross-Site Scripting (XSS)	CWE-79	User input is displayed on the page without output encoding, enabling attackers to inject malicious scripts.

44 Exploitation Scenarios: Describe how an attacker could perform:

- A **SQL Injection** attack.
 - A **Cross-Site Scripting (XSS)** attack.
- Provide example payloads for each.

SQL Injection Attack

Example Payload:

```
sql
' OR '1'='1
```

Scenario: An attacker enters this payload into the “name” field. The query becomes:

```
sql
SELECT * FROM guestbook WHERE name = ' OR '1'='1';
```

This returns all records or bypasses authentication, exposing sensitive data.

Cross-Site Scripting (XSS) Attack

Example Payload:

```
html
<script>alert('Hacked!')</script>
```

Scenario: When the application displays the message without encoding, the browser executes the script, allowing the attacker to run arbitrary JavaScript — stealing cookies or defacing the page.

45 Remediation: Rewrite the PHP code to securely handle user input, preventing both SQL Injection and XSS. Use prepared statements and appropriate output encoding.

To prevent both SQL Injection and XSS, securely handle user input using **prepared statements** and **output encoding**:

```
php
<?php
// Secure guestbook.php
include("db_config.php");

// Use prepared statements
$stmt = $conn->prepare("INSERT INTO guestbook (name, message) VALUES (?, ?)");
$stmt->bind_param("ss", $name, $message);

// Validate and sanitize input
$name = htmlspecialchars(trim($_POST['name']), ENT_QUOTES, 'UTF-8');
$message = htmlspecialchars(trim($_POST['message']), ENT_QUOTES, 'UTF-8');

$stmt->execute();
$stmt->close();

// Safely display output
$result = $conn->query("SELECT name, message FROM guestbook");
while ($row = $result->fetch_assoc()) {
    echo "<p><strong>" . htmlspecialchars($row['name'], ENT_QUOTES, 'UTF-8') . "</strong>: " .
        htmlspecialchars($row['message'], ENT_QUOTES, 'UTF-8') . "</p>";
}
$conn->close();
```

?>

Secure Practices:

- Use prepared statements for all database queries.
- Apply output encoding (`htmlspecialchars`) before displaying user data.
- Validate input length and format.
- Use parameterized queries and least privilege database accounts.

6.3 Python Code Review: Command Injection

Consider a Python Flask application that allows an administrator to ping a network host.

```
from flask import Flask, request, render_template_string

import subprocess

app = Flask(__name__)

@app.route("/ping_host", methods=["POST"])
def ping_host():
    host = request.form.get("host")
    # Vulnerable: Direct use of user input in shell command
    command = "ping -c 4 " + host
    try:
        output = subprocess.check_output(command, shell=True, text=True)
        return render_template_string(f"<pre>{output}</pre>")
    except subprocess.CalledProcessError as e:
        return render_template_string(f"<pre>Error: {e.output}</pre>")

if __name__ == "__main__":

    app.run(debug=True)
```

Practical Tasks:

46 Identify Vulnerability: What is the primary security vulnerability in this `ping_host` function? Reference the relevant CWE.

The primary vulnerability in the `ping_host` function is Command Injection. If the code executes system commands using unsanitized user input (e.g., `os.system(f"ping {host}")` or `subprocess.run("ping " + host, shell=True)`), attackers can inject arbitrary commands.

Relevant CWE: CWE-78 (Improper Neutralization of Special Elements used in an OS Command).

Nature: The application trusts user input and passes it directly to the operating system shell, allowing execution of unintended commands.

47 Exploitation Scenario: Describe how an attacker could execute arbitrary commands on the server.

An attacker could exploit this by submitting a malicious payload in the host field. **Example Payload:**

bash

127.0.0.1 && rm -rf /

Scenario: When the application runs ping 127.0.0.1 && rm -rf /, the shell executes both commands - first pinging localhost, then deleting files on the server. This results in remote code execution or system compromise.

48 Remediation: Modify the Python code to prevent command injection. (Hint: Avoid shell=True and pass arguments as a list).

To prevent command injection, avoid using shell=True and pass arguments as a list to subprocess.run(). Validate and sanitize user input before execution.

Secure Code Example:

Python

```
import subprocess
```

```
import re
```

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route('/ping')
```

```
def ping_host():
```

```
    host = request.args.get('host', '')
```

```
    # Validate input: allow only IP addresses or domain names
```

```
    if not re.match(r'^[a-zA-Z0-9.-]+$', host):
```

```
        return "Invalid host input", 400
```

```
    # Safe subprocess call without shell=True
```

```
    result = subprocess.run(['ping', '-c', '4', host], capture_output=True, text=True)
```

```
    return f"<pre>{result.stdout}</pre>"
```

```
if __name__ == '__main__':
```

```
    app.run(debug=False)
```

Secure Practices:

- Use parameterized command arguments, not shell concatenation.
- Validate input with regex or whitelisting.
- Run commands with least privilege.
- Disable debugging in production.

Part 7: Error Handling and Logging

7.1 Theory Questions

- 49** Why is it dangerous to display verbose error messages to end-users? What information might be leaked?

Displaying detailed error messages to end-users can leak sensitive internal information such as:

- Database names, table structures, or SQL queries.
- File paths and system configuration details.
- Framework versions or stack traces.
- Authentication or session tokens.

Impact: Attackers can use this information to craft targeted exploits or reconnaissance attacks.

Best Practice: Show generic user-friendly messages (e.g., "An error occurred. Please try again.") while logging detailed errors securely on the server.

- 50** Explain the principle of "Log what matters, but not too much." What kind of information should be logged, and what should be avoided?

Principle: Effective logging captures essential security-relevant events without exposing sensitive data.

Log What Matters:

- Authentication attempts (success/failure).
- Authorization changes.
- Input validation failures.
- System errors and exceptions.
- Administrative actions.

Avoid Logging:

- Passwords, session IDs, or cryptographic keys.
- Personal identifiable information (PII).
- Full database queries or raw user input.

Goal: Balance visibility for incident response with privacy and security compliance.

- 51** What is the importance of secure logging in incident response and forensics?

Purpose: Secure logging is vital for incident response and forensics because it provides reliable evidence of system activity.

Benefits:

- Enables detection and investigation of attacks.
- Supports accountability and audit trails.
- Helps reconstruct events after a breach.

Best Practices:

- Protect logs from tampering (use append-only storage or integrity checks).
- Restrict access to authorized personnel.
- Encrypt logs in transit and at rest.
- Synchronize timestamps for accurate event correlation.

7.2 PHP Code Review: Insecure Error Handling

Examine the following PHP code snippet that attempts to connect to a database.

```
<?php

// db_connect.php
$servername = "localhost";
$username = "root";
$password = "root_password"; // Hardcoded password
$dbname = "my_database";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error); // Verbose error message
}

echo "Connected successfully";

?>
```

Practical Tasks:

52 Identify Vulnerabilities: Identify at least **two** security vulnerabilities or bad practices in this code related to error handling and configuration.

Two common issues in insecure database connection code are:

1. Verbose Error Disclosure

- Example: `die("Connection failed: " . mysqli_connect_error());`
- This leaks sensitive details (database name, host, credentials, stack traces) directly to end-users.
- **Relevant CWE: CWE-209 (Information Exposure Through an Error Message).**

2. Hardcoded Credentials

- Storing the database password directly in the source code is insecure.
- If the code is leaked or misconfigured, attackers gain direct access.
- **Relevant CWE: CWE-798 (Use of Hard-coded Credentials).**

53 Remediation: Modify the PHP code to handle database connection errors securely, logging the details internally and presenting a generic message to the user. Also, address the hardcoded password issue.

The code should:

- Handle errors securely by logging details internally (e.g., to a file or monitoring system).
- Present a generic message to the user.
- Remove hardcoded passwords and load them from environment variables or secure configuration.

Secure Example:

```
php
<?php
// Load credentials securely from environment variables
$servername = getenv('DB_SERVER');
$username  = getenv('DB_USER');
$password  = getenv('DB_PASS');
$dbname    = getenv('DB_NAME');

// Attempt database connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    // Log detailed error internally
    error_log("Database connection failed: " . $conn->connect_error);

    // Show generic message to user
    die("Sorry, we are experiencing technical difficulties. Please try again later.");
}

// Continue with application logic...
?>
```

Best Practices:

- Use environment variables or a secure secrets manager for credentials.
- Log errors to a protected file or monitoring system, not to the user.
- Restrict access to logs and rotate credentials regularly.

7.3 Python Code Review: Insufficient Logging

Consider a Python Flask application where a critical operation is performed, but logging is minimal.

```
from flask import Flask, request
```

```

app = Flask(__name__)

@app.route("/process_payment", methods=["POST"])
def process_payment():
    user_id = request.form.get("user_id")
    amount = request.form.get("amount")

    # Simulate payment processing
    if float(amount) > 0:
        # Payment successful, but only minimal logging
        print(f"Payment processed for user {user_id}, amount {amount}")
        return "Payment successful!"
    else:
        # Payment failed, no specific logging
        return "Payment failed."

if __name__ == "__main__":

    app.run(debug=True)

```

Practical Tasks:

54 Identify Vulnerability: What are the shortcomings of the logging in this `process_payment` function from a security and incident response perspective? Reference the relevant CWE if applicable.

The `process_payment` function performs a critical operation but logs very little information. **Shortcomings:**

- **Lack of audit trail:** No record of who initiated the payment, when, or what the outcome was.
- **No error logging:** Failures aren't captured, making incident response and troubleshooting difficult.
- **No security context:** Missing logs for authentication or authorization checks.
- **Potential CWE: CWE-778 (Insufficient Logging)** - failure to record enough information for detecting and responding to security events.

Impact: Without detailed logs, administrators cannot detect fraud, trace malicious activity, or perform forensic analysis after an incident.

55 Remediation: Modify the Python code to implement more robust and secure logging for both successful and failed payment attempts. Ensure sensitive information is not logged directly.

Implement robust, secure logging that records meaningful events while avoiding sensitive data exposure.

Improved Code Example:

Python

```

import logging
from flask import Flask, request, jsonify

app = Flask(__name__)

# Configure secure logging
logging.basicConfig(

```

```

filename='app.log',
level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s'
)

@app.route('/process_payment', methods=['POST'])
def process_payment():
    user_id = request.form.get('user_id')
    amount = request.form.get('amount')

    try:
        # Simulate payment processing
        if not user_id or not amount:
            logging.warning(f"Payment attempt failed: missing parameters (user_id={user_id})")
            return jsonify({"error": "Invalid input"}), 400

        # Log successful transaction (without sensitive details)
        logging.info(f"Payment processed successfully for user_id={user_id}, amount={amount}")
        return jsonify({"status": "success"}), 200

    except Exception as e:
        # Log error internally, but don't expose details to user
        logging.error(f"Payment processing error for user_id={user_id}: {str(e)}")
        return jsonify({"error": "Payment failed. Please try again later."}), 500

if __name__ == '__main__':
    app.run(debug=False)

```

Secure Logging Practices:

- Log **events**, not secrets — never store card numbers, passwords, or tokens.
- Include timestamps, user identifiers, and outcomes.
- Use INFO, WARNING, and ERROR levels appropriately.
- Protect log files with access controls and integrity checks.

Part 8: Encryption and Data Protection

8.1 Theory Questions

56 When should data be encrypted at rest? When should it be encrypted in transit? Provide examples for each.

Type	When to Encrypt	Examples
Data at Rest	When data is stored on disk, in databases, backups, or any persistent medium.	Encrypt customer records, passwords, financial data, and confidential files stored on servers or cloud storage.

Data in Transit	When data is transmitted between systems, applications, or users over networks.	Use HTTPS/TLS for web traffic, VPNs for internal communication, and encrypted email protocols (e.g., TLS for SMTP).
------------------------	---	---

57 Explain the difference between **symmetric** and **asymmetric** encryption. When would you use one over the other?

Type	Description	Use Cases
Symmetric Encryption	Uses the same key for encryption and decryption. Fast and efficient but requires secure key sharing.	Encrypting large volumes of data (e.g., AES for database encryption).
Asymmetric Encryption	Uses a public key to encrypt and a private key to decrypt. Slower but enables secure key exchange.	Digital signatures, SSL/TLS handshakes, secure email (e.g., RSA or ECC).

Choice: Use **symmetric encryption** for bulk data and **asymmetric encryption** for secure key exchange or authentication.

58 What are the risks associated with using weak or outdated cryptographic algorithms?

Using obsolete algorithms (e.g., MD5, SHA-1, DES, RC4) exposes systems to:

- **Brute-force attacks:** Weak keys can be easily guessed.
- **Collision attacks:** Different inputs produce the same hash, undermining integrity.
- **Compatibility risks:** Modern libraries and compliance standards reject outdated algorithms.
- **Data breaches:** Attackers can decrypt or forge data with minimal effort.

Best Practice: Use modern, vetted algorithms such as **AES-256**, **RSA-2048**, or **SHA-256**, and follow current cryptographic standards (e.g., NIST SP 800-57).

8.2 PHP Code Review: Insecure Data Storage

Consider a PHP script that stores sensitive user data directly in a file.

```
<?php

// store_data.php
```

```
if (isset($_POST["data"])) {  
    $sensitive_data = $_POST["data"];  
    $filename = "sensitive_data.txt";  
    file_put_contents($filename, $sensitive_data . "\n", FILE_APPEND);  
    echo "Data stored.";  
} else {  
    echo "No data to store.";  
}
```

?>

Practical Tasks:

59 Identify Vulnerability: What is the major security vulnerability in this method of storing sensitive data? Reference the relevant CWE.

The major vulnerability is that sensitive user data is stored directly in a plain text file without encryption or access control. **Relevant CWE:**

- **CWE-312** - Cleartext Storage of Sensitive Information
- **CWE-522** - Insufficiently Protected Credentials

Impact:

- Anyone with file-system access can read or modify confidential data.
- Data breaches, identity theft, and compliance violations (e.g., GDPR, PCI-DSS) can occur.
- Attackers can exploit exposed credentials or personal information.

60 Remediation: Describe how this data should be securely stored, including encryption at rest.

Sensitive data should be securely stored using encryption at rest and proper access controls.

Secure Storage Practices:

1. **Use Encryption at Rest:**
 - Encrypt data before writing to disk using strong algorithms (e.g., AES-256).
 - Store encryption keys securely (environment variables, key vaults).
2. **Restrict Access:**
 - Limit file permissions to authorized users only.
 - Avoid storing sensitive data in web-accessible directories.
3. **Use Secure Storage Mechanisms:**
 - Prefer databases with built-in encryption (e.g., MySQL TDE, PostgreSQL pgcrypto).
 - For files, use PHP's `openssl_encrypt()` with a strong key.

Example Secure Code:

```
php  
<?php  
// Secure data storage example  
$data = "Sensitive user information";  
$key = getenv('ENCRYPTION_KEY'); // Load key securely from environment
```

```
// Encrypt data using AES-256-CBC
$iv = openssl_random_pseudo_bytes(openssl_cipher_iv_length('AES-256-CBC'));
$encrypted_data = openssl_encrypt($data, 'AES-256-CBC', $key, 0, $iv);

// Store encrypted data safely
file_put_contents('/secure_storage/user_data.enc', base64_encode($iv . $encrypted_data));

// Restrict file permissions
chmod('/secure_storage/user_data.enc', 0600);
?>
```

Best Practices:

- Never store raw sensitive data in plain text.
- Use strong, vetted encryption algorithms.
- Rotate encryption keys periodically.
- Implement secure backups and audit logging for access attempts.

8.3 Python Code Review: Weak Encryption Implementation

Consider a Python script that attempts to encrypt data using a weak or custom encryption method.

```
# weak_encryption.py

def encrypt_data(data, key):
    encrypted_data = ""
    for i in range(len(data)):
        encrypted_data += chr(ord(data[i]) ^ ord(key[i % len(key)]))
    return encrypted_data

def decrypt_data(encrypted_data, key):
    decrypted_data = ""
    for i in range(len(encrypted_data)):
        decrypted_data += chr(ord(encrypted_data[i]) ^ ord(key[i % len(key)]))
    return decrypted_data

sensitive_info = "My secret message"
encryption_key = "weakkey"

encrypted = encrypt_data(sensitive_info, encryption_key)
decrypted = decrypt_data(encrypted, encryption_key)

print(f"Original: {sensitive_info}")
print(f"Encrypted: {encrypted}")

print(f"Decrypted: {decrypted}")
```

Practical Tasks:

61 Identify Vulnerability: What are the significant security flaws in this custom encryption implementation? Why is it generally discouraged to implement custom cryptography?

Significant flaws in custom encryption:

- **Use of insecure algorithms:** Many custom schemes rely on simple operations like XOR, shifting, or base64 encoding - none of which provide real cryptographic security.
- **Lack of randomness:** Weak or predictable keys and initialization vectors (IVs) make ciphertexts easy to brute-force or pattern-match.
- **No authentication:** Custom encryption often omits integrity checks, allowing attackers to modify ciphertext undetected.
- **Poor key management:** Keys may be hardcoded or derived from insecure sources (like user input or timestamps).
- **No resistance to known attacks:** Without peer-reviewed design, custom algorithms are vulnerable to replay, chosen-plaintext, and differential attacks.

Why custom cryptography is discouraged:

- Cryptography is mathematically complex and easy to get wrong.
- Established libraries (like cryptography, PyCryptodome, or libsodium) are rigorously tested and maintained by experts.
- Custom implementations lack formal proofs, community vetting, and updates against new vulnerabilities.
- The CWE reference is **CWE-327: Use of a Broken or Risky Cryptographic Algorithm**.

62 Remediation: How should data be securely encrypted in Python using a well-established cryptographic library (e.g., [cryptography](#))?

Secure encryption using the `cryptography` library:

Here's a safe example using **Fernet (AES-128 in CBC mode with HMAC for authentication)**:

Python

```
from cryptography.fernet import Fernet
```

```
# Generate a secure key
key = Fernet.generate_key()
cipher = Fernet(key)
```

```
# Encrypt data
data = b"Sensitive information"
encrypted = cipher.encrypt(data)
```

```
# Decrypt data
decrypted = cipher.decrypt(encrypted)
```

```
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Best practices:

- Always use well-established libraries (cryptography, PyCryptodome, hashlib for hashing).
- Never invent your own algorithm or modify existing ones.
- Store keys securely (e.g., environment variables, key vaults).
- Include authentication (e.g., HMAC or AEAD modes like AES-GCM).
- Keep libraries updated to patch vulnerabilities.

Part 9: Secure Code Review Checklist (Advanced)

Based on all the topics covered in this lab, create a comprehensive 10-item checklist that a developer should use during a peer code review to ensure secure coding practices across various security mechanisms.

S/N	Checklist Item	Description / What to Verify
1	Input Validation	Ensure all user inputs are validated, sanitized, and type-checked before use to prevent injection attacks.
2	Authentication & Authorization	Confirm proper authentication mechanisms and verify that access controls prevent privilege escalation.
3	Cryptography	Check that only strong, proven libraries (e.g., cryptography, PyCryptodome) are used - never custom or deprecated algorithms.
4	Error Handling	Make sure error messages don't expose sensitive system details or stack traces to end users.
5	Logging & Monitoring	Verify that logs capture security events but exclude secrets, passwords, or personal data.
6	Data Protection	Ensure sensitive data is encrypted both at rest and in transit using secure protocols like TLS/HTTPS.
7	Dependency Management	Review third-party libraries for known vulnerabilities and confirm they're regularly updated.
8	Command & Query Safety	Check for injection risks (SQL, OS, LDAP) and confirm use of parameterized queries or safe APIs.
9	Session Management	Validate secure session handling - strong cookies, proper timeouts, and protection against fixation or hijacking.
10	Configuration & Deployment	Confirm debug modes are disabled, secrets aren't hardcoded, and environment settings follow least-privilege principles.

References

- [1] ICDF Academy (2024). Intro to Secure Coding [PowerPoint slides]. ICDF A Portal. Retrieved from <https://portal.icdfa.edu.ng> and Google Slides document [intro_secure_coding_20141217.pptx](#).
- [2] OpenSecurityTraining.info: [IntroSecureCoding.html](#) - Original course material
- [3] CWE - Common Weakness Enumeration: cwe.mitre.org - For CWE definitions and details
- [4] OWASP Top 10: owasp.org/www-project-top-ten/ - For common web application security risks